Tabla de contenidos

1. Iterables
    a. Iterar sobre estructuras de datos
        i. Ejemplo iterable 1
        ii. Ejemplo iterable 2
        iii. Iterador
    b. Forma básica de hacer una estructura iterable
    c. Ejemplos de iteradores personalizados
2. Generadores

Iterables

Esta semana revisaremos técnicas que diversos lenguajes, entre ellos Python, proveen para iterar (recorrer) estructuras de datos de manera sencilla y genérica. Veremos cómo implementar estos elementos y aplicar a estructuras existentes, o a estructuras creadas por nosotros.

Iterar sobre estructuras de datos

En muchas ocasiones implementaremos estructuras de datos en que resulta natural la noción de que pueden ser iteradas (recorridas). Este es el caso para estructuras que hemos visto como listas, tuplas, *sets* y diccionarios. Nos gustaría entender cómo funciona el iterar sobre estas estructuras, y aún más importante, que al momento de crear nuestras propias estructuras estas también puedan ser iteradas utilizando `for`. Para esto hay que entender dos conceptos claves: **iterable** e **iterador**.

Un **iterable** es cualquier **objeto sobre el cual se puede iterar**, es decir, que se pueden ir obteniendo paulatinamente elementos del objeto. En Python, un iterable se puede recorrer mediante el uso de `for`. En concreto, un **iterable** podría aparecer al lado derecho del `in` de un `for` (`for i in iterable:`). Estructuras *built-ins* como *sets*, listas y diccionarios, son **iterables**.

Un **iterable** se puede encontrar de las siguientes dos formas en Python:

1. **Implementado el método `__iter__()`**. Se puede iterar todas las veces que uno quiera sobre un iterable, como es el caso de las listas. Para este caso no es necesario que el objeto iterable se pueda indexar (hacer uso de `[]` para acceder a un elemento: `iterable[posición]`). Por ejemplo, los *sets* no se indexan, pero sí podemos iterar sobre ellos.
2. **Implementado el método `__getitem__()`**. El `__getitem__()` debe representar la posición de lo que se desea acceder. Cuando se pide una posición inválida se **debe** levantar una excepción de tipo `IndexError`; con eso se finalizará la iteración.

(Inicio paréntesis

Una excepción es un objeto de Python, que nos indica la existencia de un error en el código. Esto se explica con mayor detalle en los contenidos de Excepciones, puedes revisar la ruta de aprendizaje para encontrar a qué semana corresponde este contenido.

Fin paréntesis)

Ejemplo iterable 1

Implementar `__iter__()`, pero no `__getitem__()`

```
conjunto = {1, 3, 4, 6}

for i in conjunto:
    print(i, end=" ")
```

```
1 3 4 6
```

Nota: usamos `print(i, end=" ")` donde `end=" "` para que el fin de un `print` sea solo un espacio y no un salto de línea (`\n`). Así el print va hacia la derecha en vez de un número por línea.

Ejemplo iterable 2

Implementar `__getitem__()`, pero no `__iter__()`

```
class DuplicarCaracteresPalabra:
    def __init__(self, palabra_original: str) -> None:
        self._len = len(palabra_original)
        self._palabra = palabra_original

    def __getitem__(self, index: int) -> str:
        print('Obteniendo posición:', index)

        # La palabra original levanta un IndexError
        # cuando se intenta de acceder a un índice inválido
        return self._palabra[index]*2


iterable = DuplicarCaracteresPalabra('.py')
for elemento in iterable:
    print('↳', elemento)
# El for termina cuando se obtiene un IndexError,
# por lo anterior se ve que la última posición que se intentó
# de acceder no imprime
```

```
Obteniendo posición: 0
↳ ..
Obteniendo posición: 1
↳ pp
Obteniendo posición: 2
```

```
↳ yy
Obteniendo posición: 3
```

Iterador

Por otra parte, un **iterador** es un **objeto que itera sobre un iterable**, y es el objeto retornado por el método `__iter__()`. Este objeto iterador implementa el método `__next__()`, que nos retorna uno a uno los elementos de la estructura cada vez que se invoca a esta función. Cuando no quedan objetos por recorrer el iterador **debe** levantar una excepción de tipo `StopIteration`.

```
# iter(conjunto) nos entrega un objeto que itera sobre ese conjunto
conjunto = {1, 4, 6}
iterador = iter(conjunto)  # Esto es lo mismo que conjunto.__iter__()
print(type(iterador))

# Ahora vamos a invocar a next para que el iterador nos entregue
# el siguiente valor del iterable
print(next(iterador))      # Esto es lo mismo que iterador.__next__()
print(iterador.__next__())
```

```
<class 'set_iterator'>
1
4
```

Si al iterador le pedimos más elementos de los que tiene la estructura, levantará una excepción de tipo `StopIteration`.

```
print(next(iterador))
print(next(iterador))
```

```
6

---------------------------------------------------------------------------
StopIteration                             Traceback (most recent call last)
Cell In[11], line 2
      1 print(next(iterador))
----> 2 print(next(iterador))

StopIteration:
```

En resumen

> DEFINICIONES SIMPLES:
>
> *Un **iterable** es un objeto que contiene una colección de elementos y que puede ser recorrido o **iterado** uno a uno. Un **iterador** es un objeto que proporciona un*

*mecanismo para **iterar sobre un iterable**, avanzando de un elemento al siguiente hasta que se han recorrido todos los elementos.*

Un **iterable** se puede utilizar/recorrer múltiples veces; en cambio un **iterador** solo puede utilizarse una única vez.

Esto se debe a que un **iterable** tiene el método `__iter__` implementado, y este **siempre** retorna un iterador.

Por su parte, un **iterador** es un objeto que tiene el método `__next__` implementado, es decir puedo hacer `next(iterador)` y esto retornará un **valor**. Cuando el iterador se queda sin elementos que recorrer, levanta una excepción.

Forma básica de hacer una estructura iterable

Ahora veremos la forma más básica de crear un iterable e iterador que nos permita recorrer una lista de números. Para esto crearemos nuestras propias **clases iteradora e iterable**.

Nota: Hay más formas de hacer que una estructura sea iterable sin crear una nueva clase, entre ellas una que veremos más adelante.

Empezamos creando una clase iterable (`IterableListaNumeros`), la cual implementará el método `__iter__`.

```
from __future__ import annotations

# Declaramos la clase IteradorListaNumeros,
# pero no la definimos. Esto lo haremos
# mas adelante
class IteradorListaNumeros:
    pass

class IterableListaNumeros:
    def __init__(self, objeto: list) -> None:
        self.objeto = objeto

    def __iter__(self) -> IteradorListaNumeros:
        return IteradorListaNumeros(self.objeto)
```

Ahora creamos una clase iterador (`IteradorListaNumeros`), la cual será la encargada de iterar sobre el iterable.

Esta clase debe implementar:

- El método `__next__` que retornará los valores hasta que el iterador se queda sin elementos.
- El método `__iter__` que sólo debe retornar una referencia a la instancia de iterador (`self`).

Nota: Es posible no implementar el método `__iter__` en un iterador, pero no será posible obtener el iterador de forma manual (`iterador = iter(iterable)`) y luego usar el iterador directamente en el `for`.

Importante: En general, los iteradores no se pueden reiniciar.

```
# Es necesario tener una versión de
# Python >= 3.11.x para que esto funcione
# Si no le funciona, actualice su versión
from typing import Self


class IteradorListaNumeros:
    def __init__(self, iterable: list) -> None:
        # Hacemos una copia del iterable para no afectar los valores
originales
        self.iterable = iterable.copy()

    def __iter__(self) -> Self:
        return self

    def __next__(self) -> int:
        if not self.iterable:
            # Levantamos una excepción del tipo StopIteration
            # con el mensaje "Llegamos al final".
            raise StopIteration("Llegamos al final")
        else:
            valor = self.iterable.pop(0)
            return valor
```

Ahora vamos a generar nuestro iterable con los datos de una lista.

```
datos = [1, 2, 3, 4, 5]
iterable = IterableListaNumeros(datos)
for i in iterable:
    print(i, end=" ")
```

```
1 2 3 4 5
```

Recordemos que un iterable se puede iterar las veces que uno quiera. Si volvermos a hacer el `for`:

```
for i in iterable:
    print(i, end=" ")
```

```
1 2 3 4 5
```

Por otro lado, si usamos un **iterador**, este solo funcionará una vez.

```
iterador = IteradorListaNumeros(datos)
# Primer intento
print(f'Primer intento:')
for i in iterador:
```

```
    print(i, end=" ")
print('\n')

# Segundo intento con el mismo iterador
print(f'Segundo intento:')
for i in iterador:
    print(i, end=" ")
```

```
Primer intento:
1 2 3 4 5

Segundo intento:
```

Por este motivo, cada vez que queramos tener una conjunto de datos iterable, tenemos que construir un `IterableListaNumeros` cuyo método `__iter__` retorne un nuevo `IteradorListaNumeros`. De este modo, cada vez que hagamos `for`, se retorna un nuevo `IteradorListaNumeros`.

Ahora, vemos un nuevo caso: **si el iterador no implementa `__iter__` ni `__getitem__`**

```
from __future__ import annotations


class IterableListaNumerosMalo:

    def __init__(self, objeto: list) -> None:
        self.objeto = objeto

    def __iter__(self) -> IteradorListaNumerosMalo:
        return IteradorListaNumerosMalo(self.objeto)


class IteradorListaNumerosMalo:

    def __init__(self, iterable: list) -> None:
        self.iterable = iterable.copy()

    def __next__(self) -> int:
        if not self.iterable:
            raise StopIteration("Llegamos al final")
        else:
            valor = self.iterable.pop(0)
            return valor

iterable = IterableListaNumerosMalo(datos)
iterador = iter(iterable)
for i in iterador:
    print(i, end=" ")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[17], line 3
      1 iterable = IterableListaNumerosMalo(datos)
      2 iterador = iter(iterable)
----> 3 for i in iterador:
      4     print(i, end=" ")

TypeError: 'IteradorListaNumerosMalo' object is not iterable
```

Esto ocurre porque `IteradorListaNumerosMalo` no tiene `__iter__` ni `__getitem__` y por lo tanto, para el `for` esta clase no es **iterable**.

Por lo tanto, con esto podemos notar que un **iterador** es un **iterable** en sí mismo; es decir, es un tipo de **iterable**

Cómo se mencionó anteriormente, un iterador solo se puede recorrer una vez. La ventaja de esto es que podemos iterrumpir el recorrido y luego continuar desde el punto en que lo dejamos:

```
iterable = IterableListaNumeros(datos)

iterador = iter(iterable)
for i in iterador:
    print(i, end=" ")
    if i >= 3:
        break

1 2 3

for i in iterador:
    print(i, end=" ")

4 5
```

Para empezar nuevamente a iterar, debemos **obtener otro iterador**. El código de abajo funciona, debido a que cuando se invoca otra vez la función `__iter__` del iterable debido al `for`, se retorna un **nuevo iterador**.

```
for i in iterable:
    print(i, end=" ")

1 2 3 4 5
```

Cada iterador tiene su propia memoria sobre cuál es el siguiente valor a acceder, la cual **no depende del iterable**. Para verlo, creamos dos iteradores:

```
iterador_1 = iter(iterable)
iterador_2 = iter(iterable)
```

Utilizamos el primero sólo tres veces:

```
for i in iterador_1:
    print(i, end=" ")
    if i >= 3:
        break
```

```
1 2 3
```

Y el segundo iterador recorre la colección otra vez, independiente de lo recorrido por el primero.

```
for i in iterador_2:
    print(i, end=" ")
```

```
1 2 3 4 5
```

Pero si volvemos a recorrer la estructura con el primer iterador (que sólo usamos tres veces), continuaremos desde donde lo dejamos:

```
for i in iterador_1:
    print(i, end=" ")
```

```
4 5
```

Por último, una vez que el iterador agotó la estructura de datos no lo podemos utilizar de nuevo.

```
for i in iterador_1:
    print(i, end=" ")
```

Ejemplos de iteradores personalizados

Un iterador no solo nos permite recorrer los elementos de un objeto iterable, sino que también permite definir la forma en que estos elementos serán recorridos.

```
datos = [5, 1, 3, 2, 4]
```

Actualmente podemos según el orden propio del iterable:

```
from typing import Self


class IteradorListaNumerosOriginal:
    def __init__(self, iterable: list) -> None:
        self.iterable = iterable.copy()
```

```
    def __iter__(self) -> Self:
        return self

    def __next__(self) -> int:
        if not self.iterable:
            raise StopIteration("Llegamos al final")
        else:
            valor = self.iterable.pop(0)
            return valor


iterable = IteradorListaNumerosOriginal(datos)
for i in iter(iterable):
    print(i, end=" ")
```

```
5 1 3 2 4
```

Pero nos podemos asegurar de recorrer los elementos de forma ordenada:

```
from typing import Self


# Aprovechamos la herencia para no declarar las funciones que se
mantienen
class IteradorListaNumerosOrdenada(IteradorListaNumerosOriginal):
    def __iter__(self) -> Self:
        # Ordenamos los elementos del iterable antes de empezar a
recorrerlos
        self.iterable.sort()
        return self

iterable = IteradorListaNumerosOrdenada(datos)
for i in iter(iterable):
    print(i, end=" ")
```

```
1 2 3 4 5
```

O de recorrer los elementos de forma aleatoria:

```
from random import shuffle
from typing import Self


class IteradorListaNumerosAleatoria(IteradorListaNumerosOriginal):
    def __iter__(self) -> Self:
        # Mezclamos los elementos del iterable antes de empezar a
recorrerlos
        shuffle(self.iterable)
        return self
```

```
iterable = IteradorListaNumerosAleatoria(datos)
for i in iter(iterable):
    print(i, end=" ")
```

```
3 1 5 2 4
```

Adicionalmente, los iterables e iteradores no aplican únicamente para listas y números, también los podemos utilizar sobre otras estructuras de datos (tuplas, *sets*, etc.) y cualquier dato que estas contengan (*strings*, clases, etc.).

Por ejemplo, podemos recorrer un *set* que contiene *strings* y hacerlos de forma ordenada en base al largo de cada texto:

```
from typing import Self


def ordenar_por_largo(valor: str) -> int:
    return len(valor)

class IteradorSetStringsOrdenado:
    def __init__(self, iterable: str) -> None:
        self.iterable = iterable.copy()

    def __iter__(self) -> Self:
        self.iterable = list(self.iterable)
        self.iterable.sort(key=ordenar_por_largo)
        return self

    def __next__(self) -> str:
        if not self.iterable:
            raise StopIteration("Llegamos al final")
        else:
            valor = self.iterable.pop(0)
            return valor

trabalenguas = '''tres tristes tigres
tragaban trigo en un trigal
en tres tristes trastos
tragaban trigo tres tristes tigres'''.replace('\n', ' ').split(' ')
trabalenguas = set(trabalenguas)

iterable = IteradorSetStringsOrdenado(trabalenguas)
for i in iter(iterable):
    print(i, end=" ")
```

```
en un tres trigo tigres trigal tristes trastos tragaban
```

Generadores

Los **generadores** son un caso especial de los **iteradores**. Los generadores nos permiten iterar sobre secuencias de datos sin la necesidad de almacenarlos en alguna estructura especial, evitando el uso innecesario de memoria.

Una vez que terminamos de iterar sobre un generador, el generador desaparece. Esto es muy útil cuando queremos realizar cálculos sobre secuencias de números que sólo nos sirven para ese cálculo en particular. La sintaxis para crear generadores es muy parecida a la comprensión de listas, sólo que en vez de paréntesis cuadrados `[]` usamos paréntesis normales `()`.

Por ejemplo, crearemos un generador para los números pares del 0 al 18:

```
# Por el sólo hecho de usar paréntesis estamos creando un generador.
generador_pares = (2 * i for i in range(10))

print(type(generador_pares))
```

```
<class 'generator'>
```

Ahora, vamos a mostrar lo que nos entrega con un `for`. Esto es posible ya que **los generadores implementan `__iter__`** retornando `self`.

```
print(f'Recorremos el generador:')
for i in generador_pares:
    print(i, end=" ")
```

```
Recorremos el generador:
0 2 4 6 8 10 12 14 16 18
```

Nuevamente, como los generadores son un caso especial de iteradores, estos solo se pueden usar vez. Si queremos volver a acceder a los datos que el generador contenía, tendremos que crear otro generador con la misma información.

```
print(f'Recorremos el generador:')
for i in generador_pares:
    print(i, end=" ")
```

```
Recorremos el generador:
```

Recordar también que se puede usar `next`:

```
generador_pares = (2 * i for i in range(10))
print(next(generador_pares))
print(next(generador_pares))
```

```
0
2
```

⚠ **Nota importante**

En los generadores podemos usar condiciones de dos formas:

- Si usamos `if` al final, estamos **filtrando elementos**, es decir, solo se generan aquellos que cumplen la condición.
- Si usamos `if-else` dentro del generador, estamos **transformando los valores**, es decir, todos los elementos se generan, pero algunos pueden modificarse.

 Idea clave:
El `if` al final **filtra**, mientras que el `if-else` inline **transforma**.

A continuación, puedes revisar algunos ejemplos de código para ver la diferencia.

```
# Filtra: solo deja pasar valores menores a 3
generador = (i for i in range(10) if i < 3)

for x in generador:
    print(x, end=" ")
```

```
0 1 2
```

```
# Transforma: si i < 3 mantiene i, si no reemplaza por 3
generador = (i if i < 3 else 3 for i in range(10))

for x in generador:
    print(x, end=" ")
```

```
0 1 2 3 3 3 3 3 3 3
```

Ahora, veamos cuánta memoria ocupa nuestro generador, *versus* una lista que contiene todos los resultados. Para ello ocuparemos la función `getsizeof`.

```
from sys import getsizeof

generador_pares = (2 * i for i in range(10))
lista_pares = [2 * i for i in range(10)]

print("Bytes del generador:", getsizeof(generador_pares))
print("Bytes de la lista:", getsizeof(lista_pares))
```

```
Bytes del generador: 104
Bytes de la lista: 184
```

Esta diferencia es más grande si aumentamos la cantidad de resultados:

```
generador_pares = (2 * i for i in range(10 ** 6))
lista_pares = [2 * i for i in range(10 ** 6)]

print("Bytes del generador:", getsizeof(generador_pares))
print("Bytes de la lista:", getsizeof(lista_pares))
```

```
Bytes del generador: 104
Bytes de la lista: 8448728
```

Concluimos que una ventaja de los generadores es que consumen mucho menos memoria, ya que **generan** cada nuevo elemento de la secuencia cuando se le solicita, y no mantienen todos los elementos de la secuencia en memoria. Es particularmente útil cuando queremos leer archivos con muchos datos. En lugar de usar una instrucción `archivo.readlines()` para leer todos los datos de una sola vez en memoria (supongamos que queremos leer un archivo de 1GB), podemos abrir el archivo, y usar un generador para extraer una línea a la vez y así evitar llenar la memoria. En la semana de **Programación funcional** nos adentraremos más en el tema y aprenderemos a construir funciones generadoras que abra un archivo, y entregue una línea del archivo cada vez.